



Sistema de Monitoreo de Aproximación y Alerta Perimetral
SMAAP



30 DE AGOSTO DE 2026

ALUMNO CAPOUILLIEZ

A00831-2@capouilliez.edu.gt

V00368-2@capouilliez.edu.gt

E01086-1@capouilliez.edu.gt

CONTENIDO

Introducción.....	4
PROBLEMÁTICA Y SOLUCIÓN TECNOLÓGICA	6
1.1 problemática de seguridad y aproximación en IRTRA Petapa.....	6
1.2 Limitaciones de un sistema de supervisión convencional.....	6
1.3 Propuesta del Sistema SMAAP	7
1.4 Automatización aplicada a la seguridad.....	7
ARDUINO Y PROGRAMACIÓN.....	10
2.1 Arduino UNO como unidad de control	10
2.2 Entradas y salidas del sistema	10
2.3 Programación en C++ para Arduino	11
2.4 Algoritmos y estructuras condicionales.....	12
SENSOR ULTRASÓNICO Y MEDICIÓN DEL NIVEL	14
3.1 Funcionamiento del sensor HC-SR04	14
3.2 Pines Trig y Echo.....	14
3.3 Medición de distancia mediante programación.....	15
3.4 Umbral de 200 centímetros.....	15
SISTEMA DE alerta perimetral.....	18
4.1 sensor pir.....	18
4.2 led rojo y buzzer	18
4.3 prioridad de la alarma	19
4.4 Flujo de funcionamiento del sistema.....	20
INNOVACIÓN E IMPACTO DEL PROYECTO	22
5.1 aplicación de las TICS	22
5.2 Monitoreo en tiempo real	22
5.3 Apoyo al operador	22
Referencias	27

INTRODUCCIÓN

INTRODUCCIÓN

Actualmente, la tecnología y la automatización permiten desarrollar soluciones para mejorar la seguridad y eficiencia en diferentes espacios donde existe una gran cantidad de personas. En lugares recreativos como IRTRA Petapa, la seguridad de los visitantes y trabajadores es un aspecto fundamental, especialmente en las atracciones que utilizan recorridos continuos o vías de desplazamiento.

En atracciones como el Tren, los Troncos o la Montaña Rusa, los operadores necesitan conocer con anticipación cuándo un vehículo se aproxima al área final del recorrido. En determinadas zonas, la visibilidad puede ser limitada debido a curvas, estructuras u otros elementos, dificultando conocer con exactitud la posición del vehículo y preparar oportunamente el área de desembarque.

Además, las vías de aproximación pueden presentar puntos ciegos en los cuales una persona podría ingresar accidentalmente a una zona de exclusión mientras la atracción se encuentra en movimiento. Esta situación representa un riesgo importante para la seguridad.

A partir de esta problemática surge SMAAP, Sistema de Monitoreo de Aproximación y Alerta Perimetral, un prototipo que combina sensores, señales luminosas y una alarma sonora para proporcionar información al operador sobre dos situaciones diferentes: la aproximación del vehículo y la posible presencia de una persona dentro de la zona de seguridad.

El sistema utiliza un Arduino UNO como unidad de control, un sensor ultrasónico HC-SR04 para determinar la distancia de aproximación, un sensor PIR para detectar movimiento, un LED RGB para representar los estados de aproximación, un LED verde para indicar una condición segura, un LED rojo y un buzzer para emitir una alerta, además de un botón para reiniciar la alarma.

De esta manera, SMAAP busca demostrar cómo la programación y la electrónica pueden combinarse para desarrollar una solución tecnológica orientada a la supervisión, señalización y prevención de riesgos.

PROBLEMÁTICA Y SOLUCIÓN TECNOLÓGICA

PROBLEMÁTICA Y SOLUCIÓN TECNOLÓGICA

1.1 PROBLEMÁTICA DE SEGURIDAD Y APROXIMACIÓN EN IRTRA PETAPA

Uno de los principales problemas identificados para el desarrollo de SMAAP está relacionado con las atracciones que funcionan mediante recorridos continuos o sobre rieles.

En este tipo de atracciones, el operador necesita conocer la ubicación del vehículo durante las diferentes etapas del recorrido. Sin embargo, existen sectores donde la visibilidad directa puede verse limitada, especialmente cuando el vehículo se aproxima a una curva final o a la zona de llegada.

Esta falta de visibilidad puede dificultar la preparación del área de desembarque. Si el operador no conoce con suficiente anticipación que el vehículo está próximo a llegar, puede disminuirse la eficiencia del ciclo de operación.

Además del problema de aproximación, existe una situación de mayor importancia relacionada con la seguridad. Las vías de aproximación pueden contar con puntos ciegos o zonas donde una persona podría ingresar accidentalmente mientras la atracción está funcionando.

Por esta razón, consideramos necesario desarrollar un sistema que permita detectar la aproximación del vehículo y, al mismo tiempo, identificar movimiento dentro de una zona de exclusión.

1.2 LIMITACIONES DE UN SISTEMA DE SUPERVISIÓN CONVENCIONAL

En un sistema convencional, el operador depende principalmente de la observación visual y de los procedimientos establecidos para determinar cuándo una atracción se encuentra próxima a una determinada zona.

Aunque la supervisión humana es fundamental, existen situaciones en las cuales la visibilidad puede verse limitada.

Una persona puede encontrarse fuera del campo visual directo del operador o un vehículo puede aproximarse desde una zona que no pueda observarse completamente.

SMAAP propone complementar la supervisión humana mediante sensores electrónicos.

El objetivo no es sustituir al operador, sino proporcionarle información adicional mediante señales visuales y sonoras.

1.3 PROPUESTA DEL SISTEMA SMAAP

A partir de la problemática anterior se propone **SMAAP: Sistema de Monitoreo de Aproximación y Alerta Perimetral**.

El sistema utiliza dos mecanismos principales de detección.

El primero corresponde al **sensor ultrasónico HC-SR04**, encargado de medir la distancia existente entre el sensor y el objeto que se aproxima.

En la simulación se establece un límite de **200 centímetros**:

- Distancia **mayor a 200 cm**: estado de espera.
- Distancia **menor o igual a 200 cm**: vehículo próximo.

El segundo mecanismo utiliza un **sensor PIR**, encargado de detectar movimiento dentro de la zona de exclusión.

Cuando el PIR detecta movimiento, Arduino activa el estado de alarma. En ese momento:

- Se apaga el indicador **verde**.
- Se apaga el LED RGB.
- El LED **rojo** comienza a parpadear.
- El buzzer emite una señal sonora.
- La alarma permanece activa hasta que el operador presiona el botón de reinicio.

De esta manera, el sistema integra **monitoreo de aproximación + detección de intrusión + señalización visual + alerta sonora**.

1.4 AUTOMATIZACIÓN APLICADA A LA SEGURIDAD

La automatización permite que el sistema supervise continuamente las condiciones establecidas en el programa.

El proceso puede resumirse de la siguiente manera:

Medir → Detectar → Comparar → Señalizar → Alertar

Arduino recibe información de los sensores y determina qué salida debe activar.

Si no existe movimiento peligroso, el sistema puede mostrar el estado de aproximación mediante el LED RGB.

Si el vehículo está lejos, se muestra el estado amarillo.

Si el vehículo se encuentra dentro de los 200 cm establecidos, el LED RGB cambia a azul.

Por otro lado, si el sensor PIR detecta movimiento, el sistema da prioridad a la condición de alarma.

ARDUINO Y PROGRAMACIÓN

ARDUINO Y PROGRAMACIÓN

2.1 ARDUINO UNO COMO UNIDAD DE CONTROL

El **Arduino UNO** constituye la unidad principal del sistema SMAAP.

Puede considerarse como el cerebro del prototipo, ya que recibe información proveniente de los sensores y, mediante el programa desarrollado en C++, determina qué acciones deben realizar los diferentes dispositivos.

En este proyecto Arduino recibe información de:

- Sensor ultrasónico HC-SR04.
- Sensor PIR.
- Push Button.

Y controla:

- LED rojo.
- LED verde.
- LED RGB.
- Buzzer.

Por lo tanto, Arduino permite coordinar todos los componentes del sistema.

2.2 ENTRADAS Y SALIDAS DEL SISTEMA

De acuerdo con el código utilizado, las conexiones principales son:

Componente	Pin Arduino	Función
LED rojo de alarma	D11	Señal visual de peligro
RGB rojo	D10	Parte roja del LED RGB
RGB azul	D9	Parte azul del LED RGB
LED verde	D8	Indicación de zona segura
RGB verde	D6	Parte verde del LED RGB
Push Button	A1	Reinicio de alarma
Buzzer	D13	Alerta sonora
Trig HC-SR04	A2	Activación del sensor ultrasónico
Echo HC-SR04	A3	Recepción de la señal ultrasónica
PIR	A4	Detección de movimiento

Esta distribución corresponde directamente a las definiciones realizadas al inicio del programa

2.3 PROGRAMACIÓN EN C++ PARA ARDUINO

El programa está desarrollado en **C++ para Arduino**.

Primero se definen los pines que serán utilizados por cada componente:

```
</> C++  
  
#define LEDR 11  
#define RGB_ROJO 10  
#define RGB_AZUL 9  
#define LEDV 8  
#define RGB_VERDE 6  
#define Button A1  
#define Buzzer 13  
#define Trig A2  
#define Echo A3  
#define PIR A4
```

También se crean variables para almacenar información durante la ejecución:

```
</> C++  
  
bool alarmaActivada = false;  
int distancia = 0;  
long duracion = 0;  
int DetectMovi = 0;
```

La variable **alarmaActivada** permite determinar si el sistema se encuentra en estado de alarma.

La variable **distancia** almacena la distancia calculada por el sensor ultrasónico.

La variable **duracion** almacena el tiempo que tarda en regresar el pulso ultrasónico.

Finalmente, **DetectMovi** almacena el estado obtenido del sensor PIR.

2.4 Algoritmos y estructuras condicionales

El programa utiliza estructuras condicionales if para tomar decisiones dependiendo de la información obtenida por los sensores.

La lógica principal es:

¿El botón fue presionado?

→ Sí: reiniciar la alarma.

¿El PIR detectó movimiento?

→ Sí: activar alarma.

¿La alarma está activada?

→ Sí: LED rojo + buzzer intermitente.

→ No: continuar con la supervisión de aproximación.

¿La distancia es mayor a 200 cm?

→ Sí: LED RGB amarillo.

¿La distancia es menor o igual a 200 cm?

→ Sí: LED RGB azul.

Esto permite que el Arduino tome decisiones automáticamente.

SENSOR ULTRASÓNICO Y GESTIÓN DE APROXIMACIÓN

SENSOR ULTRASÓNICO Y MEDICIÓN DEL NIVEL

3.1 FUNCIONAMIENTO DEL SENSOR HC-SR04

El **HC-SR04** es un sensor ultrasónico utilizado para medir distancias.

Su funcionamiento consiste en enviar una señal ultrasónica mediante el pin **Trig** y esperar su regreso mediante el pin **Echo**.

En SMAAP, el sensor se utiliza como un sistema de medición de proximidad.

El objetivo es determinar si el vehículo se encuentra lejos de la zona final o si ya ingresó a la zona considerada próxima.

3.2 PINES TRIG Y ECHO

En el circuito desarrollado:

- **Trig** → **A2**
- **Echo** → **A3**

El programa genera primero el pulso de activación:

```
</> C++  
  
digitalWrite(Trig, LOW);  
delayMicroseconds(2);  
digitalWrite(Trig, HIGH);  
delayMicroseconds(10);  
digitalWrite(Trig, LOW);
```

Después se utiliza:

```
</> C++  
  
duracion = pulseIn(Echo, HIGH);
```

para obtener el tiempo que tarda en recibirse la señal.

3.3 MEDICIÓN DE DISTANCIA MEDIANTE PROGRAMACIÓN

Después de obtener la duración del pulso, el programa calcula la distancia:

```
</> C++  
distancia = duracion / 58;
```

El resultado se interpreta en centímetros.

Posteriormente, el valor es mostrado mediante el monitor serial:

```
</> C++  
Serial.print("La distancia de las manos al sensor es de: ");  
Serial.print(distancia);  
Serial.println(" cm");
```

Esto permite observar durante la simulación qué distancia está detectando el sensor.

3.4 UMBRAL DE 200 CENTÍMETROS

Para SMAAP se establece como referencia una distancia de **200 cm**. Cuando:

distancia > 200 cm

el sistema considera que el vehículo todavía se encuentra fuera de la zona de aproximación inmediata.

En este caso, el LED RGB muestra:

Amarillo = Estado de espera

Cuando:

distancia ≤ 200 cm

el sistema considera que el vehículo ya se encuentra próximo.

En este caso:

Azul = Vehículo próximo

Esto permite que el operador pueda interpretar visualmente la distancia sin necesidad de observar directamente el sensor o el monitor serial.

SISTEMA DE ALERTA PERIMETRAL

SISTEMA DE ALERTA PERIMETRAL

4.1 SENSOR PIR

El segundo sistema de detección utiliza un sensor **PIR**.

El sensor PIR permite detectar movimiento dentro del área supervisada.

En el código se obtiene su estado mediante:

```
</> C++  
  
DetectMovi = digitalRead(PIR);
```

Posteriormente se comprueba:

```
</> C++  
  
if (DetectMovi == HIGH) {  
    alarmaActivada = true;  
}
```

Cuando el PIR detecta movimiento, la variable `alarmaActivada` cambia a `true`.

Esto provoca que el sistema entre en estado de alarma.

4.2 LED ROJO Y BUZZER

Cuando la alarma está activada, el sistema apaga los indicadores correspondientes a la condición segura y activa los dispositivos de advertencia.

El LED rojo se enciende:

```
</> C++  
  
digitalWrite(LED_R, HIGH);
```

y posteriormente se apaga:

```
</> C++  
  
digitalWrite(LED_R, LOW);
```

Este proceso se repite para producir un parpadeo.

Al mismo tiempo, el buzzer produce una señal:

```
</> C++  
  
tone(Buzzer, 1000);
```

y posteriormente:

```
</> C++  
  
noTone(Buzzer);
```

Por lo tanto, la alerta es tanto **visual como sonora**.

4.3 PRIORIDAD DE LA ALARMA

Una característica importante del sistema es que la alarma tiene prioridad sobre la indicación de aproximación.

Cuando `alarmaActivada` es verdadera, el programa no continúa mostrando los estados normales del LED RGB.

En cambio, ejecuta el bloque:

```
</> C++  
  
if (alarmaActivada) {
```

y activa la secuencia de advertencia.

Esto permite que una posible intrusión sea tratada como una condición de mayor prioridad que el estado normal de aproximación.

4.4 FLUJO DE FUNCIONAMIENTO DEL SISTEMA

El funcionamiento completo de SMAAP puede resumirse mediante una secuencia de pasos que permite comprender cómo interactúan los sensores, Arduino y los dispositivos de señalización.

1. El sistema inicia.
2. Arduino configura los pines de entrada y salida.
3. El sensor ultrasónico HC-SR04 realiza una medición.
4. Arduino calcula la distancia en centímetros.
5. El sensor PIR comprueba si existe movimiento.
6. Si el PIR detecta movimiento, se activa la alarma.
7. El LED rojo comienza a parpadear.
8. El buzzer emite una señal sonora intermitente.
9. Si no existe movimiento, el sistema mantiene encendido el LED verde de zona segura.
10. Arduino analiza la distancia obtenida por el sensor ultrasónico.
11. Si la distancia es **mayor a 200 cm**, el LED RGB muestra **amarillo**, indicando estado de espera.
12. Si la distancia es **menor o igual a 200 cm**, el LED RGB muestra **azul**, indicando que el vehículo se encuentra próximo.
13. El sistema continúa realizando mediciones constantemente.
14. Si la alarma se encuentra activa, el operador puede presionar el Push Button para reiniciarla una vez que la zona haya sido despejada.

De esta forma, el sistema combina la detección de aproximación y la detección de movimiento en un mismo circuito. Arduino procesa continuamente la información obtenida de los sensores y determina qué señal visual o sonora debe activarse.

INNOVACIÓN E IMPACTO DEL PROYECTO

INNOVACIÓN E IMPACTO DEL PROYECTO

5.1 APLICACIÓN DE LAS TICS

SMAAP integra diferentes elementos tecnológicos:

Electrónica + sensores + programación + automatización + seguridad.

Los sensores permiten obtener información del entorno, mientras que Arduino procesa esa información y genera respuestas mediante luces y sonidos.

Esto demuestra que las TICS pueden utilizarse no solamente para almacenar o transmitir información, sino también para **detectar condiciones físicas y responder automáticamente ante ellas.**

5.2 MONITOREO EN TIEMPO REAL

Una de las características principales del proyecto es que el sistema realiza mediciones constantemente.

El sensor ultrasónico proporciona información sobre la aproximación y el sensor PIR permite identificar movimiento.

Arduino procesa estos datos continuamente y actualiza las señales del sistema.

Por esta razón, el prototipo puede considerarse un sistema de **monitoreo en tiempo real dentro de la simulación.**

5.3 APOYO AL OPERADOR

SMAAP busca proporcionar al operador información adicional mediante señales fáciles de interpretar.

El código establece tres situaciones principales:

Verde: zona segura.

Amarillo: vehículo fuera de la zona de aproximación inmediata.

Azul: vehículo próximo, a 200 cm o menos.

Rojo + Buzzer: movimiento detectado y alarma activada.

Esto permite que diferentes condiciones sean identificadas rápida

CONCLUSIONES

El proyecto **SMAAP: Sistema de Monitoreo de Aproximación y Alerta Perimetral** surgió a partir de una problemática relacionada con la supervisión de atracciones de recorrido continuo y la seguridad de las zonas próximas a las vías de circulación.

Mediante la utilización de un Arduino UNO, un sensor ultrasónico HC-SR04 y un sensor PIR, desarrollamos un sistema capaz de supervisar dos condiciones diferentes: la aproximación de un vehículo y la detección de movimiento dentro de una zona de exclusión.

El sensor ultrasónico permite medir la distancia y compararla con un límite establecido de 200 centímetros. Cuando el vehículo se encuentra fuera de esta distancia, el sistema muestra un estado amarillo, mientras que cuando se aproxima a 200 centímetros o menos, el LED RGB cambia a azul.

Por otra parte, el sensor PIR permite detectar movimiento. Cuando se identifica una posible intrusión, Arduino activa una alarma mediante un LED rojo intermitente y un buzzer, generando una advertencia visual y sonora.

El Push Button permite al operador reiniciar la alarma después de comprobar que la zona se encuentra despejada.

Desde el punto de vista de TICS, SMAAP demuestra cómo la combinación de **hardware y software** puede utilizarse para desarrollar sistemas automatizados de supervisión. Los sensores proporcionan información, Arduino procesa los datos y los dispositivos de salida comunican el resultado.

Finalmente, el proyecto demuestra que mediante componentes relativamente sencillos es posible desarrollar un prototipo que represente una solución tecnológica orientada a mejorar la información disponible para un operador y reforzar la supervisión de una zona de seguridad.

REFERENCIAS

REFERENCIAS

- Arduino. (s. f.). ARDUINO UNO REV3. Arduino Documentation. [Arduino UNO Rev3 – Arduino Documentation](#)
- Arduino. (s. f.). LANGUAGE REFERENCE. Arduino Documentation. [Arduino Language Reference](#)
- Elec freaks. (s. f.). HC-SR04 ULTRASONIC SENSOR. Elec freaks. [HC-SR04 Ultrasonic Sensor](#)
- Arduino Project Hub. (s. f.). PIR MOTION SENSOR. Arduino Project Hub. [Arduino Project Hub](#)
- Arduino. (s. f.). BUILT-IN EXAMPLES. Arduino Documentation. [Arduino Built-in Examples](#)
- Código **y circuito del prototipo SMAAP**. (2026). Elaboración propia realizada en Arduino/Tinkercad.